

CACHING — PRODUCTION LEVEL CODE EXAMPLES & BEST PRACTICES

(Node.js + Express + Redis + MongoDB / Mongoose example)

1. SETUP

```
npm install express mongoose ioredis dotenv
```

.env

```
MONGO_URI=mongodb://localhost:27017/myapp
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
```

2. SERVER SETUP

```
const express = require('express');
const mongoose = require('mongoose');
const Redis = require('ioredis');
require('dotenv').config();

const app = express();
app.use(express.json());

// MongoDB Connection
mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => console.log('MongoDB Connected'))
.catch(err => console.log(err));

// Redis Client
const redis = new Redis({ host: process.env.REDIS_HOST, port:
process.env.REDIS_PORT });

module.exports = { app, redis };
```

3. MONGOOSE USER MODEL

```
const { Schema, model } = require('mongoose');

const userSchema = new Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: Number
}, { timestamps: true });

module.exports = model('User', userSchema);
```

4. CACHE-ASIDE EXAMPLE (READ + WRITE)

4.1 READ WITH CACHE

```
const { redis } = require('./server');
const User = require('./User');

async function getUserById(userId) {
  const cacheKey = `user:${userId}`;

  const cached = await redis.get(cacheKey);
  if (cached) {
    console.log('Cache Hit');
    return JSON.parse(cached);
  }

  console.log('Cache Miss');
  const user = await User.findById(userId);

  if (user) {
    await redis.setex(cacheKey, 300, JSON.stringify(user)); // TTL 5 min
  }

  return user;
}
```

4.2 WRITE WITH CACHE INVALIDATION

```
async function updateUser(userId, data) {  
  const cacheKey = `user:${userId}`;  
  
  const updatedUser = await User.findByIdAndUpdate(userId, data, { new: true });  
  
  await redis.del(cacheKey);  
  
  return updatedUser;  
}
```

5. WRITE-THROUGH EXAMPLE (STRONG CONSISTENCY)

```
async function writeThroughUpdate(userId, data) {  
  const cacheKey = `user:${userId}`;  
  
  await redis.set(cacheKey, JSON.stringify(data), 'EX', 300);  
  const updatedUser = await User.findByIdAndUpdate(userId, data, { new: true });  
  
  return updatedUser;  
}
```

6. WRITE-BEHIND EXAMPLE (ASYNC, HIGH THROUGHPUT)

```
async function writeBehindUpdate(userId, data) {  
  const cacheKey = `user:${userId}`;  
  
  await redis.set(cacheKey, JSON.stringify(data), 'EX', 300);  
  
  setImmediate(async () => {  
    await User.findByIdAndUpdate(userId, data);  
  });  
}
```

7. PAGINATION + CACHE KEY DESIGN

```
async function getUsers(page = 1, limit = 10) {
  const cacheKey = `users:page:${page}:limit:${limit}`;

  const cached = await redis.get(cacheKey);
  if (cached) return JSON.parse(cached);

  const users = await User.find()
    .skip((page - 1) * limit)
    .limit(limit)
    .lean();

  await redis.setex(cacheKey, 120, JSON.stringify(users));
  return users;
}
```

8. CACHE STAMPEDE PREVENTION (LOCK + RANDOM TTL)

```
const { v4: uuidv4 } = require('uuid');

async function getUserWithLock(userId) {
  const cacheKey = `user:${userId}`;
  const lockKey = `lock:${cacheKey}`;

  const cached = await redis.get(cacheKey);
  if (cached) return JSON.parse(cached);

  const lock = await redis.set(lockKey, uuidv4(), 'NX', 'EX', 5);
  if (!lock) {
    await new Promise(r => setTimeout(r, 50));
    return getUserWithLock(userId);
  }

  const user = await User.findById(userId);
  if (user) {
    const randomTTL = 300 + Math.floor(Math.random() * 30);
    await redis.setex(cacheKey, randomTTL, JSON.stringify(user));
  }

  await redis.del(lockKey);
  return user;
}
```

9. BEST PRACTICES SUMMARY

- Cache is **never source of truth**
- Use **Cache-Aside** by default
- Use **Write-Through** for strong consistency
- Use **Write-Behind** for non-critical high throughput
- Always **set TTL**
- Use **unique cache keys** (include version / params)
- Handle **cache stampede**
- Monitor **hit/miss ratios**
- Use **multi-layer caching** if needed (client, CDN, server)

10. PRODUCTION LEVEL NOTES

- Separate Redis for sessions vs caching to avoid eviction conflicts
- Monitor memory usage & evictions
- Consider Redis Cluster / Sentinel for HA & sharding
- Use compression for large payloads
- Logging cache hits/misses for analysis
- Keep DB as **source of truth**, never rely on cache for writes

 This document contains **all production-level caching patterns with full code examples** ready to use for real-world applications.